



Trabajo Práctico – Unidad 03

Verificaciones de un tablero eléctrico

Cátedra: Programación en Computación — UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván

Objetivos

Al terminar este trabajo práctico vas a poder:

- 🍌 Declarar con `const` y `let` según corresponda, y nombrar con las convenciones del curso
 - 🗃 Reconocer el **tipo** de un valor y diagnosticar con `typeof`
 - ➕ Usar los operadores aritméticos, incluido el **resto** (`%`)
 - ⚖ Escribir **comparaciones** que devuelvan `true` o `false`
 - 🔗 Combinar condiciones con `&&`, `||` y `!`
 - 🤖 Evitar la trampa de sumar textos en lugar de números
-

Consignas generales

1. **Todo se resuelve dentro del template de la cátedra.** Escribí en `src/app.js` y ejecutá con `npm run dev`.
2. **Un problema, un archivo.** Guardá cada solución aparte y copiala al `src/app.js` la que estés probando.
3. **`const` por defecto,** `let` sólo si el valor tiene que cambiar. **Nunca** `var`.
4. **Los límites y tolerancias del problema van en** `UPPER_SNAKE_CASE`, y no repetidos como números sueltos en el medio del código.
5. **Identificadores en inglés** (`nominalVoltage`, `measuredCurrent`); los textos que ve el usuario, en español y con la unidad entre corchetes.
5. **Siempre** `===` y `!==`. Usar `==` se corrige como error.
7. **Sin punto y coma** al final de las sentencias.

💡 Varios problemas piden imprimir `true` o `false`. **Eso es correcto y es el punto:** hoy las verificaciones sólo responden; en la unidad 04 van a decidir qué hace el programa.

Problemas

Problema 1 – Ficha de tablero

Declarará las siguientes variables con el tipo de declaración que corresponda y mostrará cada una junto con su tipo, usando `typeof`.

DATO	VALOR
Identificación del tablero	TAB-3
Tensión nominal [V]	380
Cantidad de ramas	6
¿Está energizado?	true
Última calibración	<i>sin cargar</i> (dejala sin asignar)

Salida esperada:

```
TAB-3 → string
380 → number
6 → number
true → boolean
undefined → undefined
```

💡 Preguntate para cada una: ¿este valor cambia durante el programa? Si no, va `const`.

Problema 2 – Relevamiento de una rama

Un técnico mide una rama del tablero. Pedí por consola la **tensión** y la **corriente**, y calculá:

- La **potencia** [W]
- La **resistencia** equivalente [Ω]
- La **potencia** que dispararía si la corriente aumentara un 20 %

Todos los valores con dos decimales.

Ejemplo de ejecución:

```
Tensión medida [V]: 380
Corriente medida [A]: 4.2

Potencia:           1596.00 W
Resistencia:        90.48  $\Omega$ 
Potencia con +20 % I: 1915.20 W
```

⚠ Las mediciones tienen decimales: `parseFloat()`, no `parseInt()`.

Problema 3 – Reparto de sensores

Se van a instalar sensores en el tablero, agrupados en módulos de igual capacidad. Pedí por consola la **cantidad de sensores** y la **capacidad de cada módulo**, y calculá:

- Cuántos módulos se llenan **por completo**
- Cuántos sensores quedan **sueltos**
- Si el reparto es **exacto** (sin sobrantes) — `true` o `false`

Ejemplo de ejecución:

```
Cantidad de sensores: 17
Capacidad por módulo: 5

Módulos completos: 3
Sensores sueltos: 2
Reparto exacto: false
```

Segunda corrida:

```
Cantidad de sensores: 20
Capacidad por módulo: 5

Módulos completos: 4
Sensores sueltos: 0
Reparto exacto: true
```

💡 Son cantidades enteras: acá sí va `parseInt()`.

💡 Los sueltos salen del **resto** (%). Para los módulos completos, restale los sueltos al total antes de dividir.

Problema 4 – Verificación de tolerancia

La norma admite una tensión de línea de **380 V con ±5 %**. Pedí por consola la tensión medida y mostrá:

- El **límite inferior** y el **superior**
- La **desviación** respecto del nominal, en porcentaje
- Si la medición está **en rango** — `true` o `false`

$$\text{desviación} = \frac{\text{medida} - \text{nominal}}{\text{nominal}} \times 100$$

Ejemplo de ejecución:

```
Tensión medida [V]: 372.5
```

```
Límites:      361.0 V a 399.0 V
```

```
Desviación: -1.97 %
```

```
En rango:    true
```

Segunda corrida:

```
Tensión medida [V]: 405
```

```
Límites:      361.0 V a 399.0 V
```

```
Desviación: 6.58 %
```

```
En rango:    false
```

⚠️ `380` y `0.05` son **constantes del problema**: van en `UPPER_SNAKE_CASE` arriba del archivo, no repetidas en cada cuenta.

💡 "En rango" son **dos** condiciones que se cumplen a la vez → `&&`.

Problema 5 – Orden de mantenimiento

Un motor entra en mantenimiento si cumple **cualquiera** de estas condiciones:

- Superó las **10.000 horas** de servicio
- La temperatura de carcasa pasó los **85 °C**
- **No** pasó la última inspección visual

Pedí por consola las horas, la temperatura y si pasó la inspección (escribiendo `true` o `false`), y mostrará cada condición por separado y el resultado final.

Ejemplo de ejecución:

```
Horas de servicio: 4200
```

```
Temperatura de carcasa [°C]: 91
```

```
¿Pasó la inspección? (true/false): true
```

```
Excede horas:      false
```

```
Excede temperatura: true
```

```
No pasó inspección: false
```

```
A mantenimiento:    true
```

⚠ **Ojo con la inspección.** `prompt()` devuelve **texto**: lo que recibís es `'true'`, no `true`. Compará contra el texto: `inspection === 'true'`.

💡 "No pasó la inspección" es la negación de "pasó" → `!`.

💡 Con que se cumpla **una** alcanza → `||`.

Problema 6 – Informe de consumo con verificaciones

Integra todo lo anterior.

Pedí por consola los datos de un motor y emití un informe. Además de los cálculos, el informe tiene que incluir **tres verificaciones**.

DATO	UNIDAD	EJEMPLO
Identificación	—	M-14
Tensión nominal	V	380
Corriente nominal	A	4.2
Factor de potencia ($\cos \phi$)	—	0.85
Horas de uso por día	h	8

Constantes del problema (van en `UPPER_SNAKE_CASE`):

- Potencia máxima admitida por la rama: **1500 W**
- Factor de potencia mínimo exigido: **0.92**
- Horas por día a partir de las cuales se considera **servicio continuo**: **12**

Calculá:

- **Potencia aparente** $S = V \times I$, en VA
- **Potencia activa** $P = S \times \cos \varphi$, en W
- **Consumo mensual** $E = \frac{P \times h \times 30}{1000}$, en kWh

Verificá:

- ¿La potencia activa **excede** el máximo de la rama?
- ¿El factor de potencia **cumple** el mínimo exigido?
- ¿Es un motor **apto sin observaciones**? — sólo si no excede la potencia **y** cumple el factor de potencia **y** no está en servicio continuo

Ejemplo de ejecución:

```
Identificación del motor: M-14
Tensión nominal [V]: 380
Corriente nominal [A]: 4.2
Factor de potencia: 0.85
Horas de uso por día: 8
```

— Informe M-14 —

```
Potencia aparente: 1596.00 VA
Potencia activa: 1356.60 W
Consumo mensual: 325.58 kWh
```

— Verificaciones —

```
Excede 1500 W: false
Cumple  $\cos \varphi \geq 0.92$ : false
Servicio continuo: false
```

```
Apto sin observaciones: false
```

💡 **Cómo encararlo:** primero los cinco `prompt()` y las tres cuentas. Verificá que los números salen bien. **Recién después** agregá las verificaciones, de a una.

💡 Guardá cada verificación en su propia variable con nombre descriptivo (`exceedsMaxPower` , `meetsPowerFactor` , `isContinuousService`). Después la verificación final se escribe combinándolas, y se lee casi como una oración.

🚫 Restricciones

- ❌ **Sin condicionales** (`if` , `switch`). Se ven en la **unidad 04**. Las verificaciones se **imprimen**, no se usan para decidir.
- ❌ **Sin bucles** (`for` , `while` , `do-while`). Unidad **05**.
- ❌ **Sin funciones propias**. Unidad **06**.
- ❌ **Sin** `var` .
- ❌ **Sin** `==` **ni** `!=` . Siempre `===` y `!==` .
- ✅ Lo de esta unidad: `const` / `let` , `typeof` , `+` `-` `*` `/` `%` `**` , `+=` , `++` , `>` `<` `>=` `<=` `===` `!==` , `&&` `||` , `!` , `parseInt` , `parseFloat` , `.toFixed()` .

😓 **Por qué el problema 6 se siente repetitivo.** Vas a escribir cinco `prompt()` casi idénticos y tres verificaciones con la misma forma. Esa incomodidad es real y tiene nombre: es lo que resuelven las **funciones** de la unidad 06. Guardate la solución: vamos a volver a este problema y lo vas a reescribir en la mitad de líneas.

Material de consulta

- [Apunte de la unidad](https://github.com/italijancic/pc-2026/blob/main/unidades/03-variables-y-operadores/apunte.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/03-variables-y-operadores/apunte.pdf)
- [Presentación de clase](https://github.com/italijancic/pc-2026/blob/main/unidades/03-variables-y-operadores/presentacion.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/03-variables-y-operadores/presentacion.pdf)
- [Ejemplos de la clase](https://github.com/italijancic/pc-2026/tree/main/unidades/03-variables-y-operadores/ejemplos) (https://github.com/italijancic/pc-2026/tree/main/unidades/03-variables-y-operadores/ejemplos)
- [Template del curso](https://github.com/italijancic/pc-2026/tree/main/template) (https://github.com/italijancic/pc-2026/tree/main/template)